Nombre: Emmanuel Carrillo Loera Edgar Daniel del Ángel Hernández Fatima Alejandra Cavazos Serrato **Equipo:** 3	**Matrícula**: 2067988 2094613 2099488
Nombre del curso: Componentes de Lógica de Negocios	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 3	**Actividad**: Revisión SOLID de actividad fundamental 3
Fecha: 13/10/2026	
Bibliografía (Formato APA) Archivo “Actividad_Unidad_3_CLN_ArquitecturaAlumnos.docx” proporcionado en la plataforma TEAMS Archivo “03_Sazón_Urbano.docx” proporcionado en la plataforma TEAMS	

Revisión con SOLID

Revisión de la propuesta completa

Principio	Pregunta propuesta	Respuesta con evidencia de nuestro diseño
S	¿Cada clase o servicio tiene una razón principal para cambiar?	Sí: tres servicios separados (`Pedido`, `Disponibilidad`, `Entrega`) y un Handler por Command. Vigilar `Pedido`, que concentra varias responsabilidades
O	¿Se puede agregar una variante de regla sin editar una cadena de condiciones?	Riesgo con **promociones**: hoy hay un solo tipo, pero la entrevista menciona un 2x1 y US-18 habla de “condiciones”
L	¿Las implementaciones se pueden sustituir sin cambiar el comportamiento?	Los repositorios en memoria y relacionales deben comportarse igual, y el contrato no estaba escrito
I	¿Cada consumidor depende solo de lo que usa?	`DisponibilidadRepository` mezclaba lectura y escritura, pero `QuitarLineaAgotada` solo necesita leer
D	¿Aplicación y dominio dependen de abstracciones?	Sí: solo interfaces (repositorios, Unit of Work, publicador, notificador, control de acceso). No hay SQL en el modelo

Qué significa cada principio y dónde se ve en nuestro diseño

- **S (responsabilidad única):** una clase debe tener una sola razón principal para cambiar. En clase vimos el ejemplo de OrderService, que solo gestiona pedidos y no envíos. En nuestro diseño hay tres Application Services separados (pedidos, disponibilidad y entregas) y un Handler por cada command. Cada uno coordina un solo caso de uso, y las reglas de negocio quedan dentro de Pedido, Promoción y Entrega.
- **O (abierto/cerrado):** se pueden agregar comportamientos nuevos sin modificar el código que ya funciona. En nuestro diseño, la Promoción calcula su propio descuento, así que el servicio y el Pedido no dependen del tipo de promoción.
- **L (sustitución de Liskov):** una implementación debe poder ocupar el lugar de otra sin cambiar el comportamiento del sistema. Aplica a los repositorios: el de memoria y uno relacional deben comportarse igual, y por eso definimos un contrato común.
- **I (segregación de interfaces):** una clase no debe depender de operaciones que no necesita. Por eso separamos las interfaces de lectura (ConsultasDe…) de los repositorios, que son los que escriben.
- **D (inversión de dependencias):** la lógica de alto nivel depende de abstracciones y no de implementaciones concretas. El dominio y la aplicación solo conocen interfaces; las implementaciones viven en infraestructura y se conectan en un solo lugar, la raíz de composición.